

Fundamentos Básicos de Sistemas Distribuídos

Aula 05

Prof. Paulo Bressan



Sumário



Centralizado, Distribuído e
Descentralizado



Aspectos importantes: acesso a recursos,
transparência, abertura e escalabilidade



Técnicas de escalabilidade



Distributed Hash Tables (DHT)

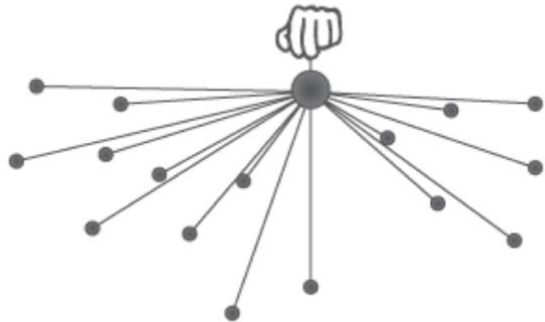


Ciladas: as falsas premissas dos
sistemas distribuídos

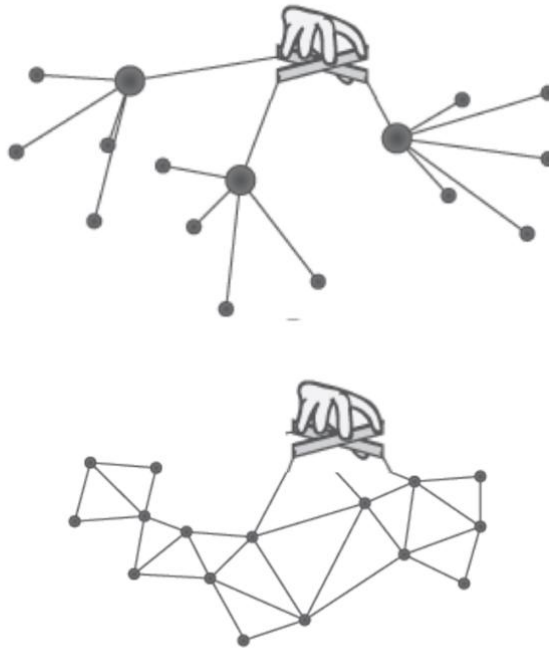


Tipos de sistemas distribuídos

Centralized system



Distributed system



Decentralized system



Notice no hand - means no central controller

Centralizado, Distribuído e Descentralizado

Quais
sistemas
existem na
Sociedade?

Como eles
se
comportam?

Quem são
beneficiados
por eles?

Quem são
prejudicados
por eles?



Definição

- Definição de Andrew Tanenbaum:

“Um sistema distribuído é um conjunto de computadores independentes que se apresenta a seus usuários como um sistema único e coerente.”

- Definição de Leslie Lamport:

"Um sistema distribuído é aquele em que a falha de um computador que você nem sabia que existia pode tornar o seu próprio computador inutilizável."

Aspectos Importantes

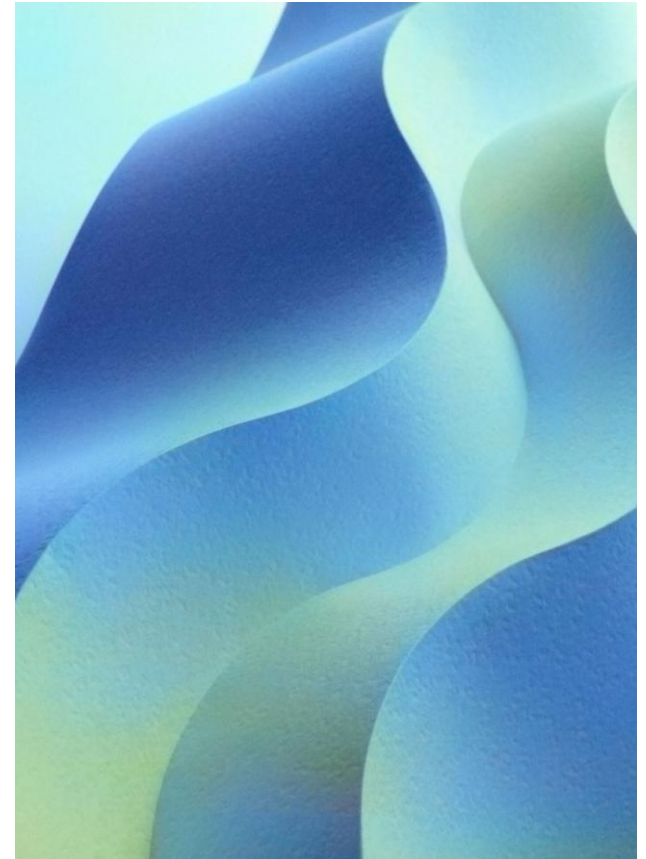
Um sistema distribuído consiste em componentes autônomos.

Usuários (pessoas ou programas) acham que estão tratando com um único sistema.

Diferenças entre componentes estão ocultas dos usuários.

O mesmo vale para sua organização interna.

Usuários e aplicações podem interagir de maneira consistente, independente de onde estejam.



Middleware

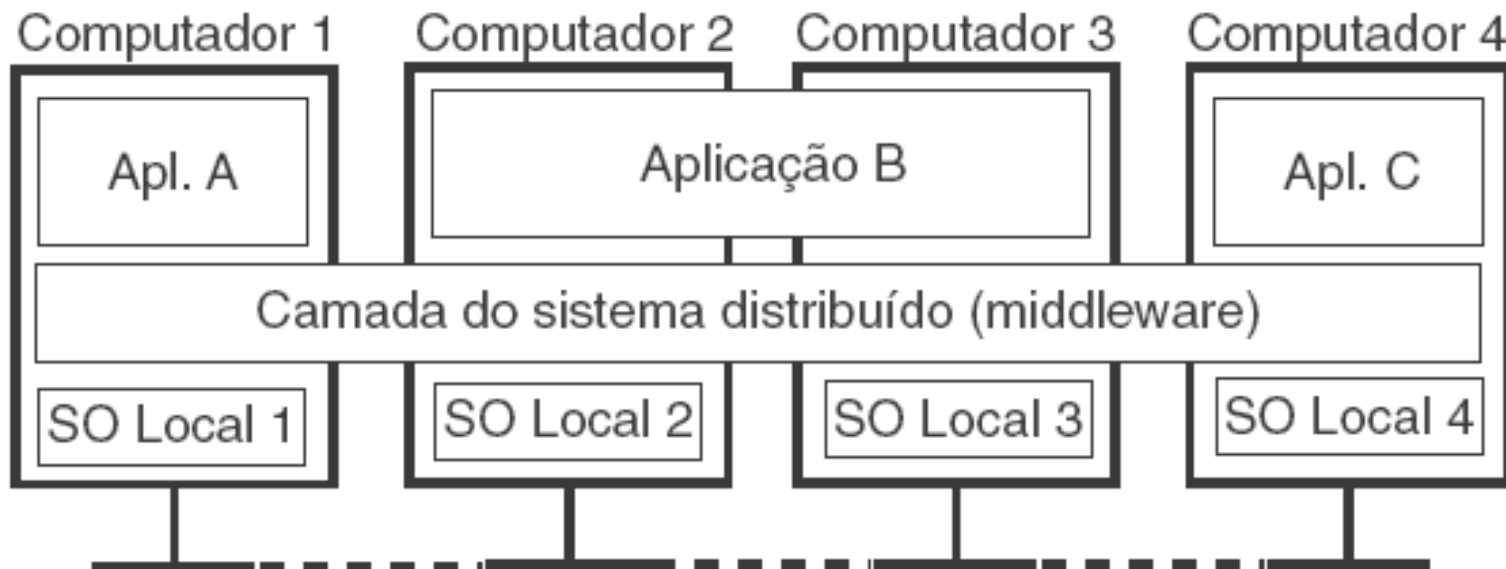


Figura 1.1 Sistema distribuído organizado como middleware.
A camada de middleware se estende por várias máquinas e oferece a mesma interface a cada aplicação.

Metas

Acesso a recursos

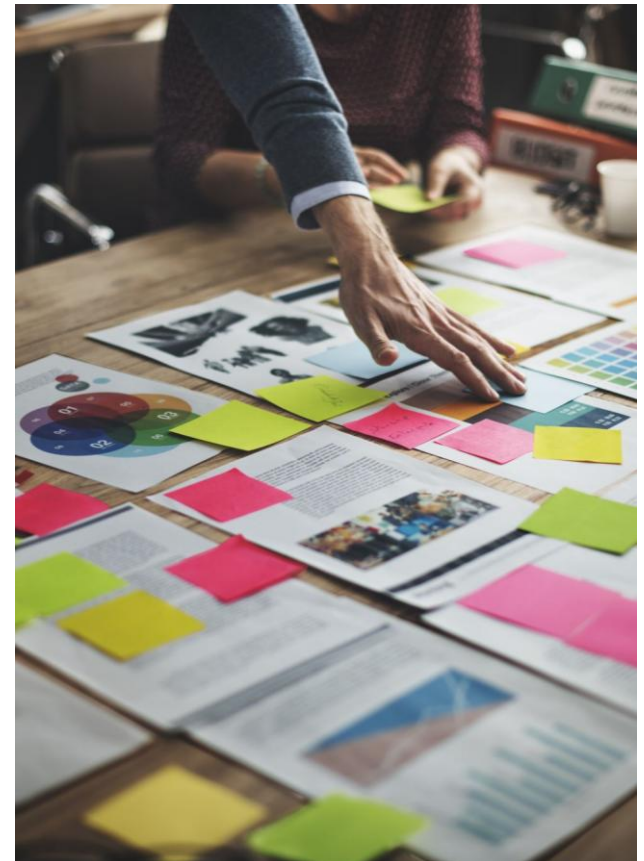
Transparência da distribuição

Abertura

Escalabilidade

Acesso a recursos

- Facilitar aos usuários e às aplicações o acesso a recursos remotos e seu compartilhamento de maneira controlada e eficiente.
- Exemplos típicos: impressoras, computadores, dados, páginas Web e redes.
- Problemas?



Transparência da distribuição

Ocultar o fato que seus processos estão fisicamente distribuídos por vários computadores.

A principal dificuldade de mascarar falhas está na incapacidade de distinguir entre um recurso morto e um recurso insuportavelmente lento.

Por exemplo, quando contactamos um servidor Web ocupado.

Transparência	Descrição
Acesso	Ocultar diferenças na representação de dados e no modo de acesso a um recurso
Localização	Ocultar o lugar em que um recurso está localizado
Migração	Ocultar que um recurso pode ser movido para outra localização
Relocação	Ocultar que um recurso pode ser movido para uma outra localização enquanto em uso
Replicação	Ocultar que um recurso é replicado
Concorrência	Ocultar que um recurso pode ser compartilhado por diversos usuários concorrentes
Falha	Ocultar a falha e a recuperação de um recurso

Tabela 1.1 Diferentes formas de transparência em um sistema distribuído (ISO, 1995).

Abertura

- Um sistema distribuído aberto é um sistema que oferece serviços de acordo com regras padronizadas que descrevem a sintaxe e a semântica desses serviços.
- Em geral, os serviços são especificados por meio de interfaces, com o uso de IDL (Interface Definition Language).
- **Interoperabilidade** caracteriza até que ponto duas implementações de sistemas ou componentes de fornecedores diferentes devem coexistir e trabalhar em conjunto, com base na confiança mútua, especificados por um **padrão comum**.

Abertura

- **Portabilidade** caracteriza até que ponto uma aplicação desenvolvida para um sistema A pode ser executada, sem modificação, em um sistema distribuído B que implementa as mesmas interfaces que A.
- Um sistema é **extensível** quando é fácil adicionar componentes ou substituir os existentes sem afetar os demais.
- É crucial que o sistema seja organizado como um conjunto de componentes relativamente pequenos e de fácil substituição ou adaptação.

Abertura (Política x Mecanismo)

- **O que é Mecanismo:** São as ferramentas ou métodos de baixo nível que implementam funcionalidades básicas do sistema.
- **Função:** Fornecem como algo pode ser feito, sem decidir qual escolha deve ser tomada.
- **Exemplo:**
 - Um mecanismo de escalonamento pode ser a **fila de processos** com prioridade.
 - O mecanismo de comunicação pode ser o **send/receive** entre processos.
 - O mecanismo de controle de acesso pode ser a **lista de permissões**.

Abertura (Política x Mecanismo)

- **O que é Política:** São as regras ou critérios que definem qual decisão tomar usando os mecanismos disponíveis.
- **Função:** Determinam o comportamento do sistema em termos de objetivos desejados.
- **Exemplo:**
 - A política de escalonamento pode ser **priorizar processos interativos** ou **distribuir igualmente entre usuários**.
 - A política de comunicação pode definir **quem pode enviar mensagens para quem**.
 - A política de controle de acesso pode dizer **quais usuários têm permissão de leitura ou escrita em um arquivo**.

Abertura (Política x Mecanismo)



Mecanismo = meios (como) → Fornece as operações fundamentais.



Política = escolhas (o quê/quem/quando /quanto) → Define os objetivos e critérios de decisão.



Por que separar os dois?

Separar mecanismo de política torna o sistema mais flexível. Assim, o mesmo mecanismo pode ser usado com políticas diferentes em contextos distintos.

Escalabilidade



é a capacidade de um sistema — seja ele um software, banco de dados, rede ou infraestrutura — de **manter ou melhorar seu desempenho** à medida que a **carga de trabalho aumenta**, seja pelo crescimento no número de usuários, no volume de dados ou na quantidade de requisições.

1. **Crescimento sustentável** → o sistema suporta expansão sem precisar ser refeito do zero.
2. **Eficiência** → o aumento de recursos (mais máquinas, mais memória, etc.) gera ganho de desempenho proporcional.
3. **Flexibilidade** → adapta-se a diferentes tipos de crescimento (mais usuários, mais dados, mais serviços).

Problemas de Escalabilidade

Conceito	Exemplo
Serviços centralizados	Um único servidor para todos os usuários
Dados centralizados	Uma única lista telefônica on-line
Algoritmos centralizados	Fazer roteamento com base em informações completas

Tabela 1.2 Exemplos de limitações de escalabilidade.

Problema de Escalabilidade

- Diferenças entre algoritmos descentralizados:
 - Nenhuma máquina tem informações completas sobre o estado do sistema
 - As máquinas tomam decisões tendo como base somente informações locais
 - A falha de uma máquina não arruína o algoritmo
 - Não há nenhuma premissa implícita quanto a existência de um relógio global.



Técnicas de Escalabilidade

1. Escalabilidade Horizontal e Vertical
 2. Técnicas de Escalabilidade Funcional
 3. Técnicas de Escalabilidade de Comunicação
 4. Técnicas de Escalabilidade de Coordenação
 5. Técnicas de Escalabilidade em Sistemas de Armazenamento
-

1. Escalabilidade Horizontal e Vertical

- **Horizontal (scale-out):** adicionar mais máquinas/servidores ao sistema.

Ex.: clusters, farms de servidores, microsserviços.

- **Vertical (scale-up):** aumentar a capacidade de uma única máquina.

Ex.: mais CPU, memória ou disco em um servidor.

2. Técnicas de Escalabilidade Funcional

a) Particionamento (Divisão do Trabalho): distribuir dados ou tarefas em diferentes nós para reduzir carga em um único ponto.

Exemplos:

- ✓ **Sharding** em bancos de dados (cada nó guarda uma parte dos dados).
- ✓ Dividir tarefas de processamento em subtarefas (MapReduce).

b) Replicação: criar cópias de dados ou serviços em vários nós para aumentar disponibilidade e desempenho.

Tipos:

- ✓ **Replicação de dados** → cópias em vários servidores para leitura rápida.
- ✓ **Replicação de serviços** → múltiplos servidores rodando o mesmo serviço.

c) Caches: manter dados frequentemente acessados em memória rápida (local ou distribuída).

Ex.: Memcached, Redis, CDNs para conteúdo web.

3. Técnicas de Escalabilidade de Comunicação

- **Multicast/Broadcast eficiente:** envio de mensagens para vários processos sem duplicação desnecessária.
- **Balanceamento de carga:** distribuir requisições de forma equilibrada entre servidores.
Ex.: DNS round-robin, load balancers (HAProxy, NGINX).
- **Redução de latência:** compressão de mensagens, protocolos eficientes (gRPC, QUIC).

Técnicas de Escalabilidade

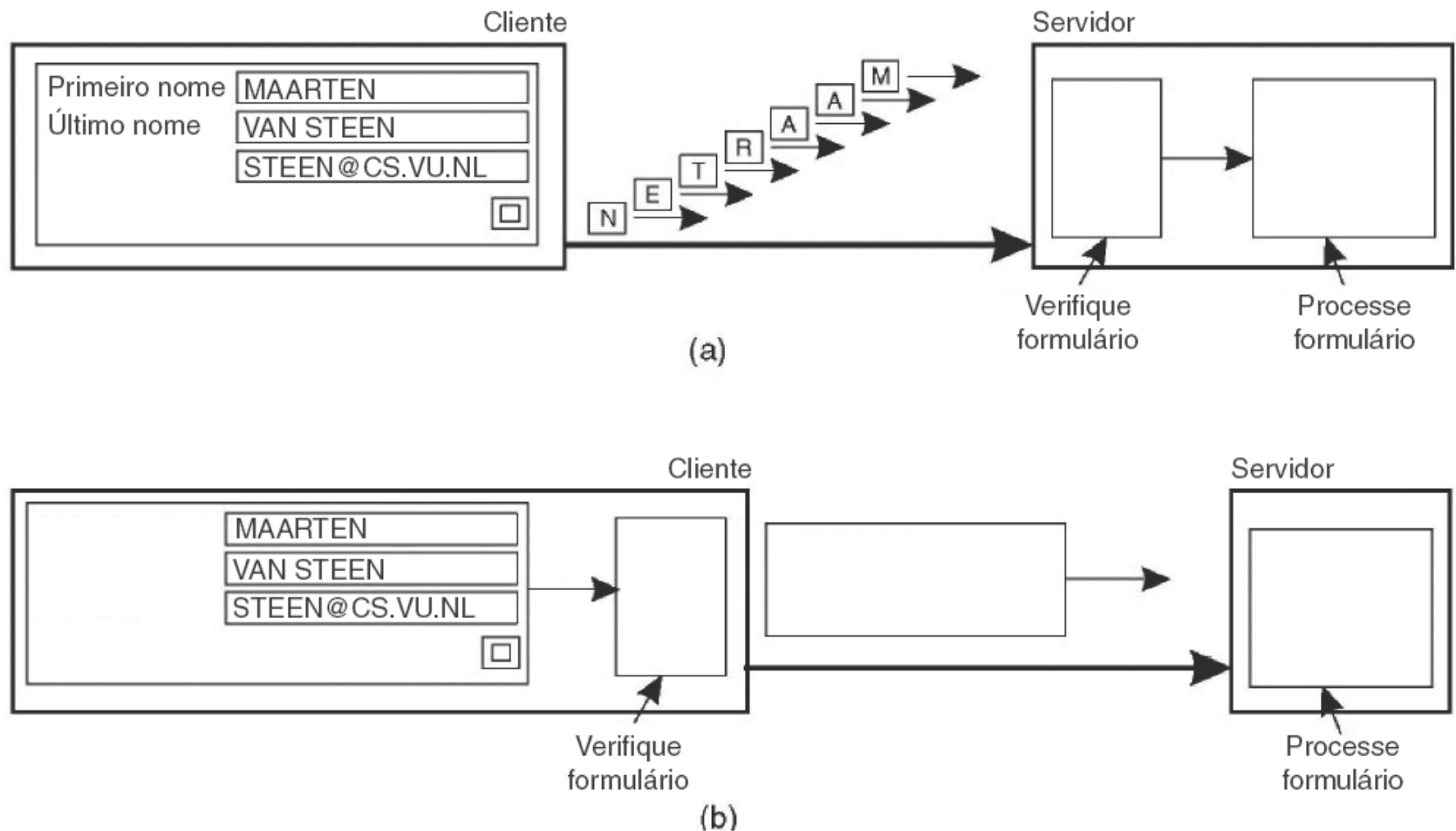


Figura 1.2 A diferença entre deixar (a) um servidor ou (b) um cliente verificar formulários à medida que são preenchidos.

4. Técnicas de Escalabilidade de Coordenação

- **Eliminação de gargalos centrais:** evitar que um único nó seja o coordenador fixo.

Ex.: algoritmos de eleição de líder (Raft, Paxos).

- **Descentralização:** estruturas P2P, sistemas baseados em DHT (como o Chord).
- **Tolerância a falhas:** replicação + consistência eventual (teorema CAP em bancos NoSQL).

5. Técnicas de Escalabilidade em Sistemas de Armazenamento

- **Sistemas de arquivos distribuídos** (Google File System, HDFS).
- **Bancos de dados distribuídos** (Cassandra, MongoDB).
- **Estruturas de indexação distribuída** para busca em grande escala (ElasticSearch).

Resumo das Técnicas

Técnica	Objetivo	Exemplos práticos
Escalabilidade Vertical (scale-up)	Aumentar capacidade de um nó	Mais CPU, RAM, SSD
Escalabilidade Horizontal (scale-out)	Adicionar nós para aumentar capacidade total	Clusters, microsserviços
Particionamento (Sharding)	Dividir dados/tarefas	MongoDB, Cassandra, MapReduce
Replicação	Aumentar disponibilidade e desempenho	PostgreSQL Replication, Cassandra
Caches	Reduzir acessos a dados lentos	Redis, Memcached, CDNs
Balanceamento de carga	Distribuir requisições entre servidores	HAProxy, NGINX, AWS ELB
Comunicação eficiente	Reduzir sobrecarga e latência	gRPC, QUIC, multicast
Descentralização / P2P	Evitar gargalos centrais	Chord, Kademlia, BitTorrent
Tolerância a falhas	Disponibilidade mesmo com falhas	Raft, Paxos, DynamoDB
Armazenamento distribuído	Gerenciar grandes volumes de dados	HDFS, S3, Ceph
Indexação distribuída	Escalar buscas em grandes bases	Elasticsearch, Solr

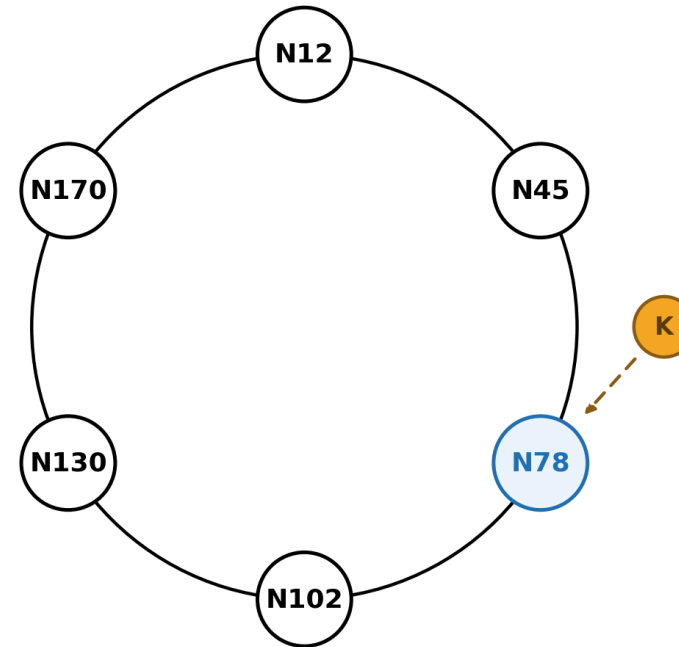
O que é uma DHT?

- Uma **Distributed Hash Table** é um mecanismo que permite **armazenar e localizar pares chave→valor** em uma rede distribuída de forma eficiente, **sem depender de um servidor central**.
- **Hash Table tradicional:** funciona em memória local → cada chave é transformada em um índice.
- **DHT:** distribui esses pares (chave, valor) entre os nós da rede → cada nó é responsável apenas por uma parte do espaço de chaves.

Como funciona uma DHT?

- **1. Identificadores:** nós e chaves recebem um ID no mesmo espaço de hash.
- **2. Nó sucessor:** cada chave é responsabilidade do primeiro nó com ID igual ou maior, em sentido horário.
- **3. Roteamento:** a requisição é encaminhada nó a nó até o responsável pela chave.
- **4. Eficiência:** tabelas de atalhos (ex.: Chord) reduzem a busca para $O(\log N)$ saltos.

Anel de hash (DHT)



cada nó é responsável pelo intervalo de chaves entre ele e seu antecessor no anel

DHT

Vantagens

- Escalabilidade (suporta milhões de nós).
- Descentralização (sem ponto único de falha).
- Balanceamento de carga automático.

Desvantagens

- Complexidade de implementação.
- Manutenção dinâmica: quando nós entram/saem da rede (churn), a DHT precisa se reorganizar.
- Segurança: ataques Sybil podem prejudicar a consistência.

As 8 falácias da computação distribuída

Peter Deutsch — Sun Microsystems

1 A rede é confiável

Pacotes se perdem, conexões caem; todo sistema precisa lidar com retransmissão e timeout.

2 A latência é zero

Toda comunicação leva tempo; uma chamada remota nunca é tão rápida quanto uma local.

3 A largura de banda é infinita

Redes têm capacidade limitada; grandes volumes de dados podem saturar links.

4 A rede é segura

Dados em trânsito podem ser interceptados sem autenticação e criptografia.

5 A topologia não muda

Nós entram, saem, IPs mudam; hardcodear a topologia quebra o sistema.

6 Há apenas um administrador

Sistemas cruzam domínios administrativos distintos, cada um com suas políticas.

7 O custo de transporte é zero

Mover dados tem custo real de processamento, infraestrutura e, na nuvem, financeiro.

8 A rede é homogênea

Protocolos, hardware e formatos variam entre nós; uniformidade não pode ser assumida.

Por que isso importa na prática

- Cada suposição incorporada implicitamente ao projeto se torna fonte recorrente de bugs, gargalos de desempenho ou falhas de segurança em produção.
- Testes locais tendem a esconder a rede real — baixa latência, sem perda de pacotes, ambiente único e controlado — e mascaram esses problemas até tarde demais.
- Conecta-se diretamente à transparência de falhas: por mais que o sistema tente mascarar a distribuição, a rede real nunca se comporta como uma extensão perfeita da memória local.

Tipos de Sistemas Distribuídos

Sistemas de Computação Distribuídos

- Sistemas de Computação de Cluster
- Sistemas de Computação em Grade

Sistemas de Informação Distribuídos

- Sistemas de Processamento de Transações
- Integração de Aplicações Empresariais

Sistemas Distribuídos Pervasivos

- Sistemas Domésticos
- Sistemas Eletrônicos para Tratamento de Saúde
- Redes de Sensores

Sistemas de Computação de Cluster

Homogeneidade

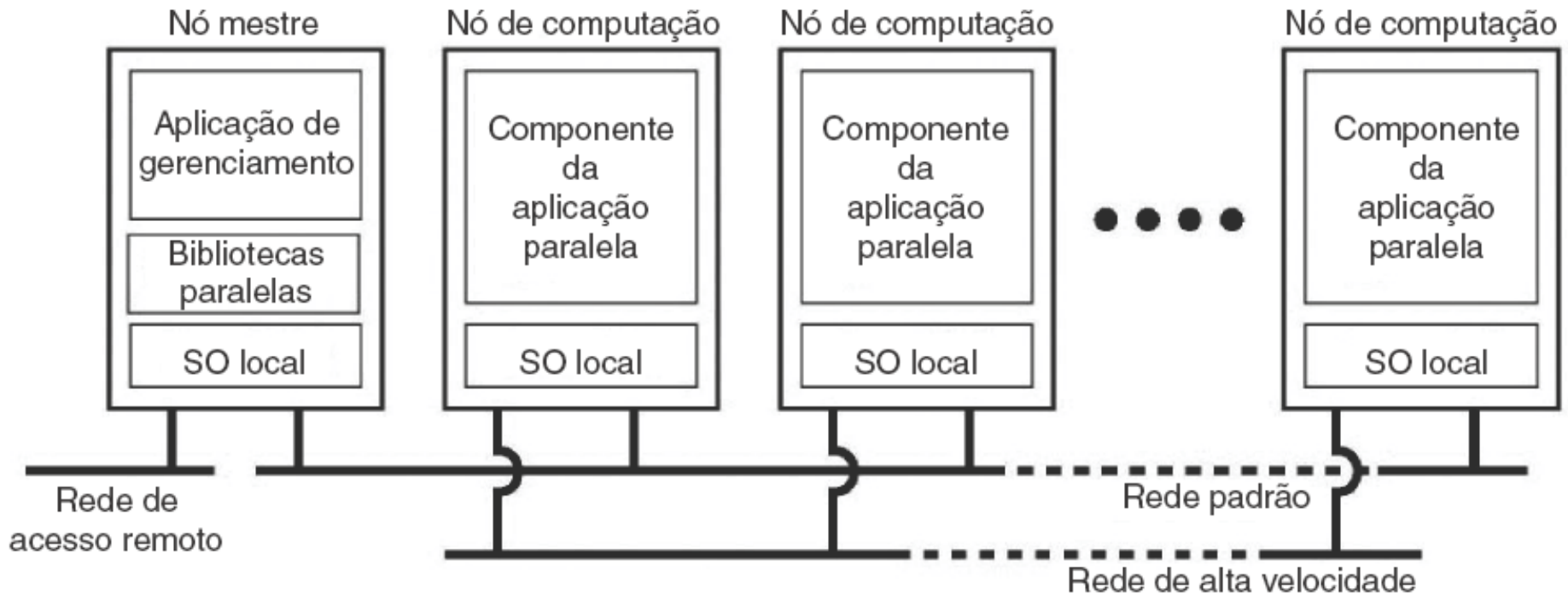


Figura 1.4 Exemplo de um sistema de computação de cluster.

Sistemas de Computação em Grade

Alto grau de heterogeneidade: organização virtual.

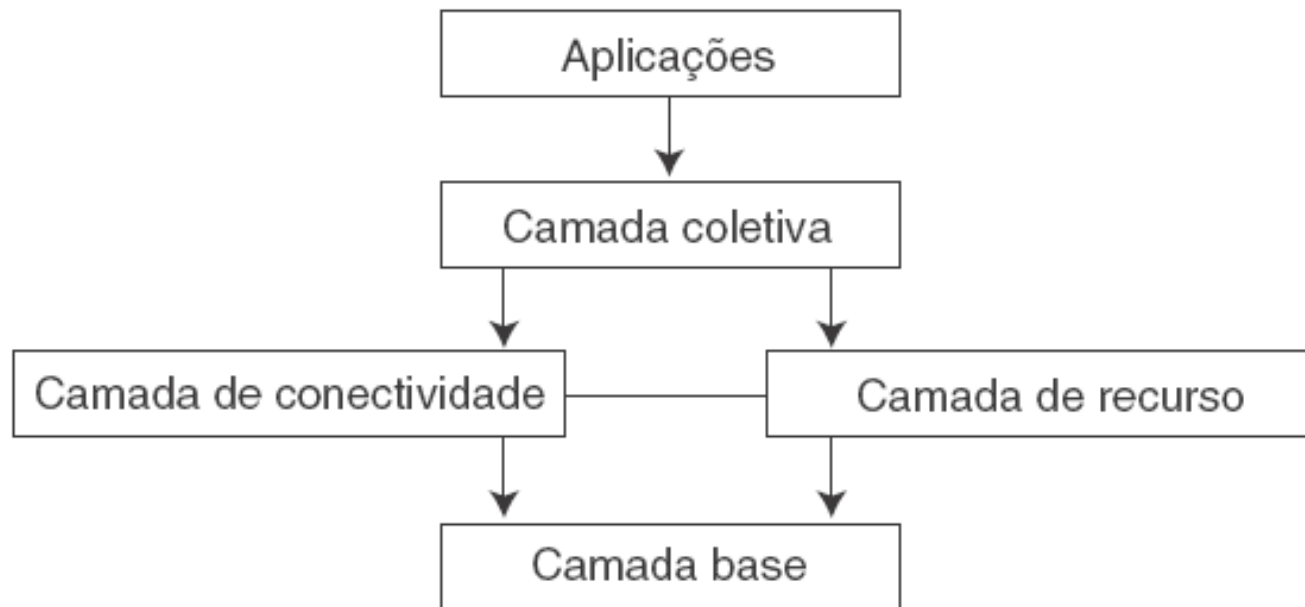


Figura 1.5 Arquitetura em camadas para sistemas de computação em grade.



Sistemas de Informação Distribuídos

- Encontrada em organizações que se defrontaram com uma profusão de aplicações em rede, onde a interoperabilidade se mostrou difícil.
- Mais fácil integrar aplicações a um sistema de informações de âmbito empresarial.

Sistemas de Processamento de Transações

Exemplo: sistema de banco de dados

Primitiva	Descrição
BEGIN_TRANSACTION	Marque o início de uma transação
END_TRANSACTION	Termine a transação e tente comprometê-la
ABORT_TRANSACTION	Elimine a transação e restaure os valores antigos
READ	Leia dados de um arquivo, tabela ou de outra forma
WRITE	Escreva dados para um arquivo, tabela ou de outra forma

Tabela 1.3 Exemplos de primitivas para transações.

Sistemas de Processamento de Transações

- 4 propriedades características:
 1. Atômicas: para o mundo exterior, a transação acontece como se fosse indivisível
 2. Consistentes: a transação não viola invariantes de sistema
 3. Isoladas: transações concorrentes não interferem umas com as outras
 4. Duráveis: uma vez comprometida uma transação, as alterações são permanentes

Sistemas de Processamento de Transações

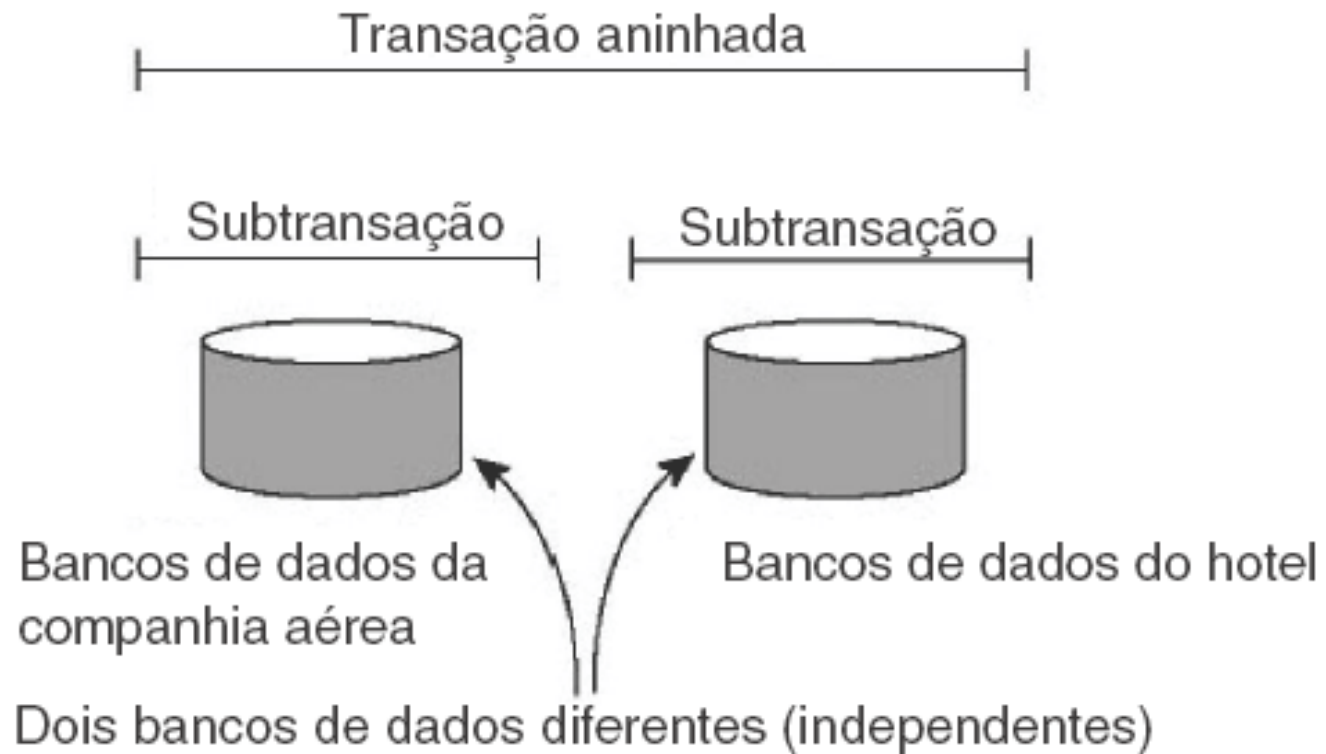


Figura 1.6 Transação aninhada.

Integração de aplicações empresariais

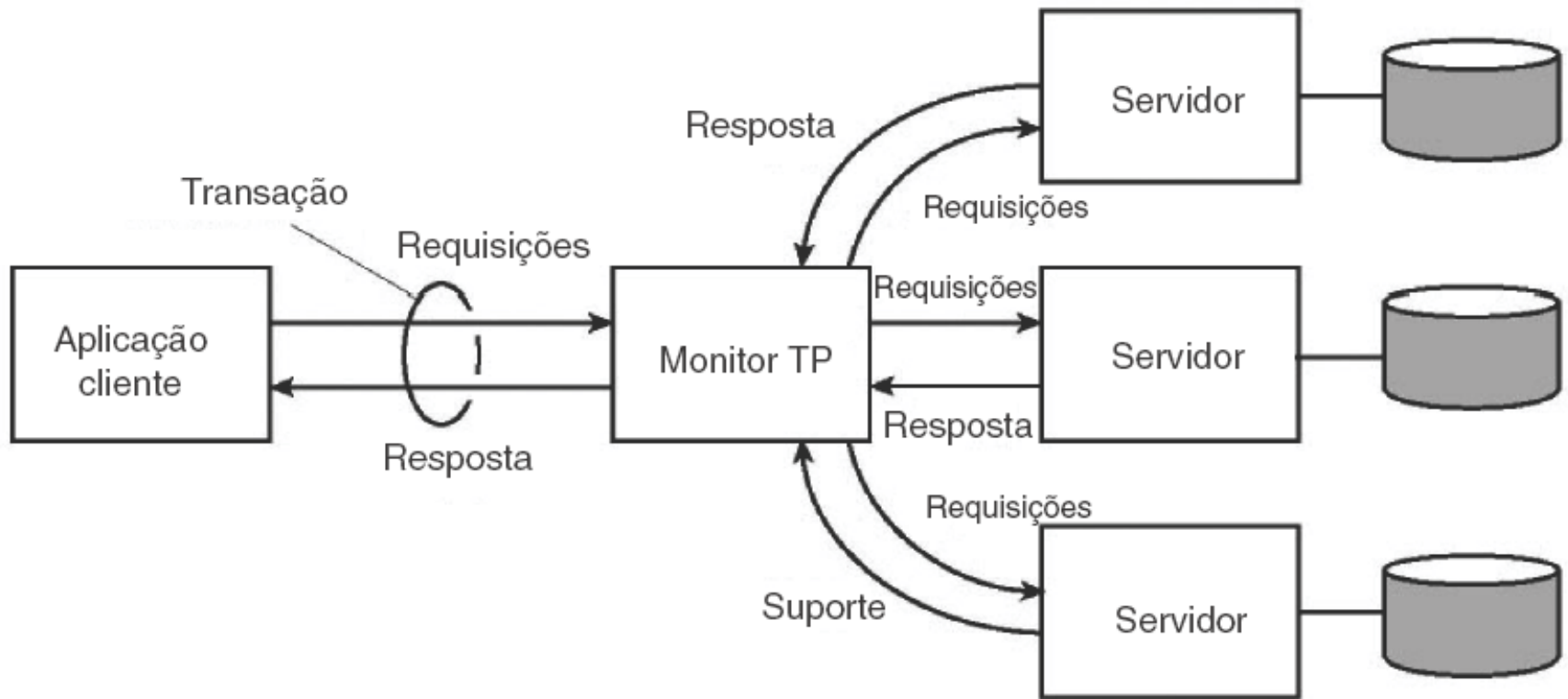


Figura 1.7 O papel do monitor TP em sistemas distribuídos.

Integração de aplicações empresariais

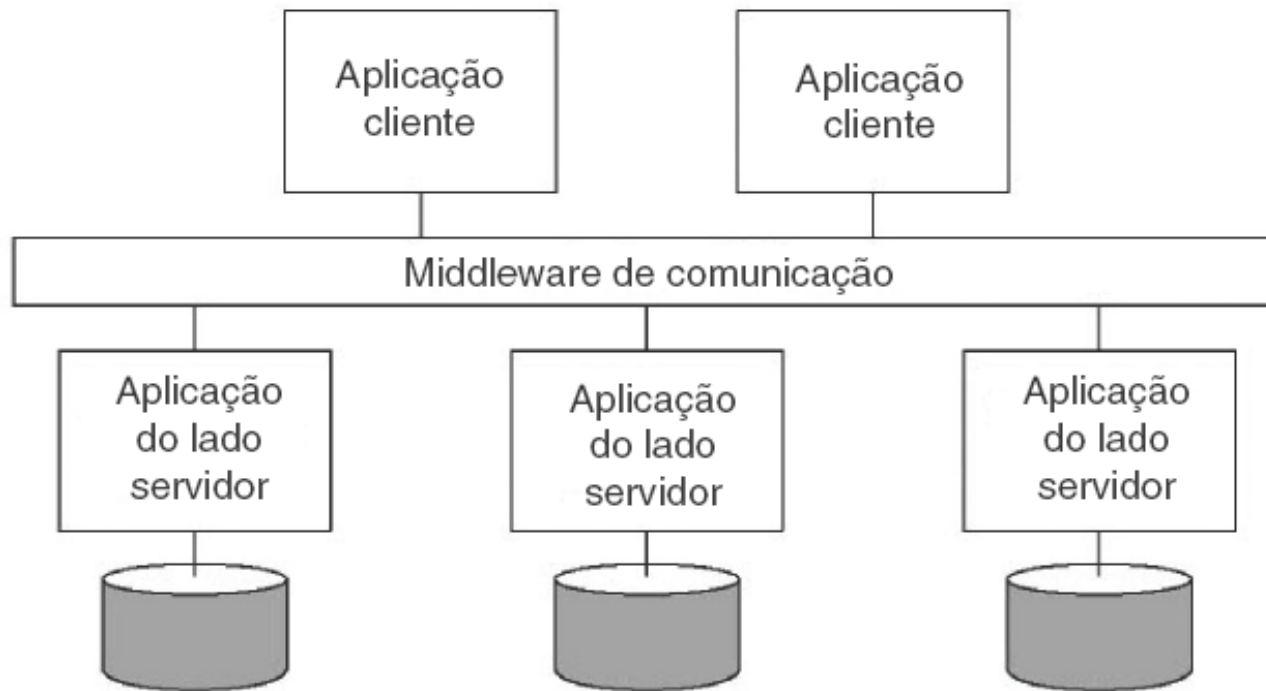
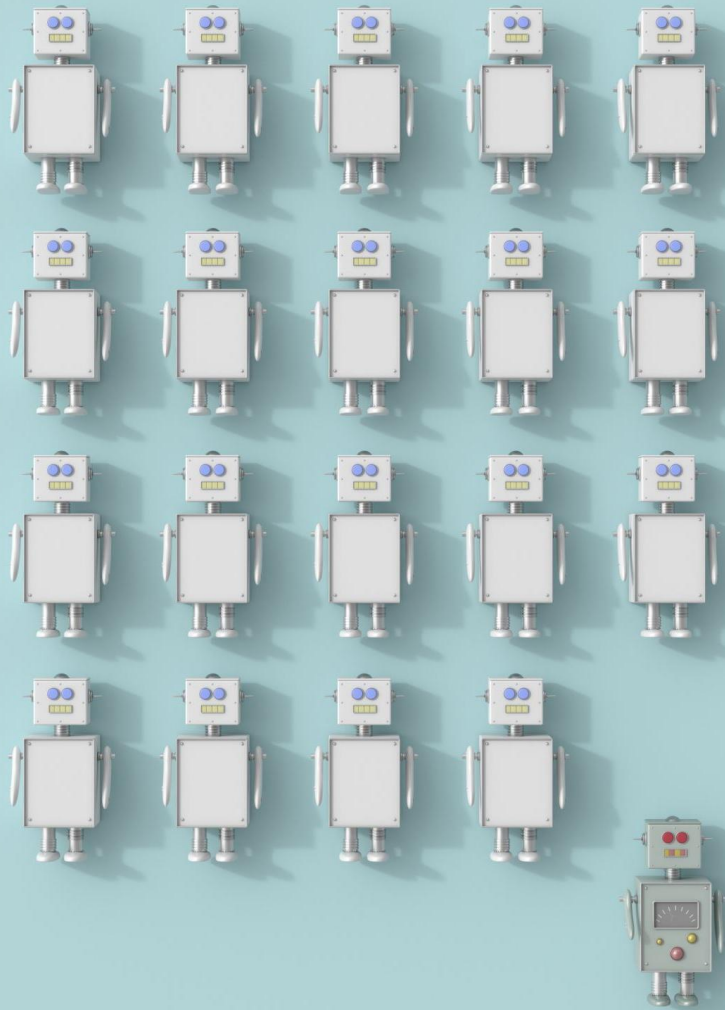


Figura 1.8 Middleware como facilitador de comunicação em integração de aplicações empresariais.

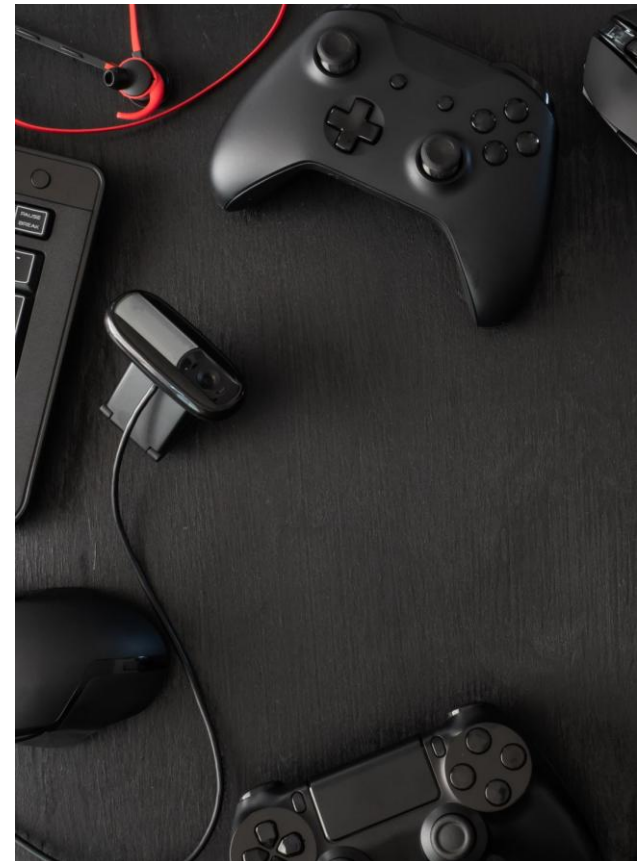


Sistemas Distribuídos Pervasivos

- Introdução de dispositivos de computação móveis e embutidos
- Os equipamentos costumam ser caracterizados por seu pequeno tamanho, pela alimentação por baterias, por sua mobilidade e pela conexão sem fio.

Sistemas Domésticos

- Ao redor de redes domésticas
 - Aparelhos de TV
 - Equipamentos de áudio e vídeo
 - Dispositivos para jogos
 - Smartphones
 - PDAs
 - E outros
- Padrões Universal Plug and Play (UPnP)



Sistemas Eletrônicos para Tratamento de Saúde

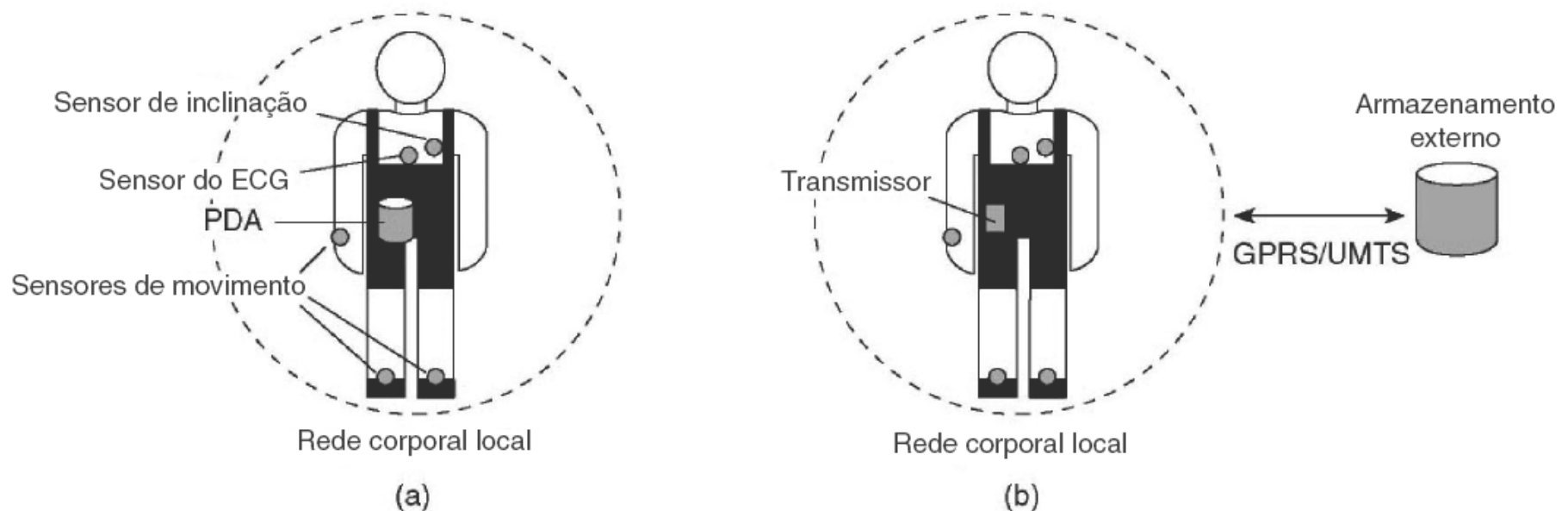


Figura 1.9 Monitoração de uma pessoa em um sistema eletrônico pervasivo de tratamento de saúde utilizando (a) um hub local ou (b) uma conexão contínua sem fio.

Sistemas Eletrônicos para Tratamento de Saúde



Onde e como os dados monitorados deverão ser armazenados?



Como podemos evitar a perda de dados cruciais?



Qual é a infraestrutura necessária para gerar e transmitir sinais de alerta?



Como os médicos podem dar retorno online?



Como pode ser alcançada a extrema robustez do sistema de monitoração?



Quais são as questões de segurança e como as políticas adequadas podem ser impostas?

Redes de Sensores

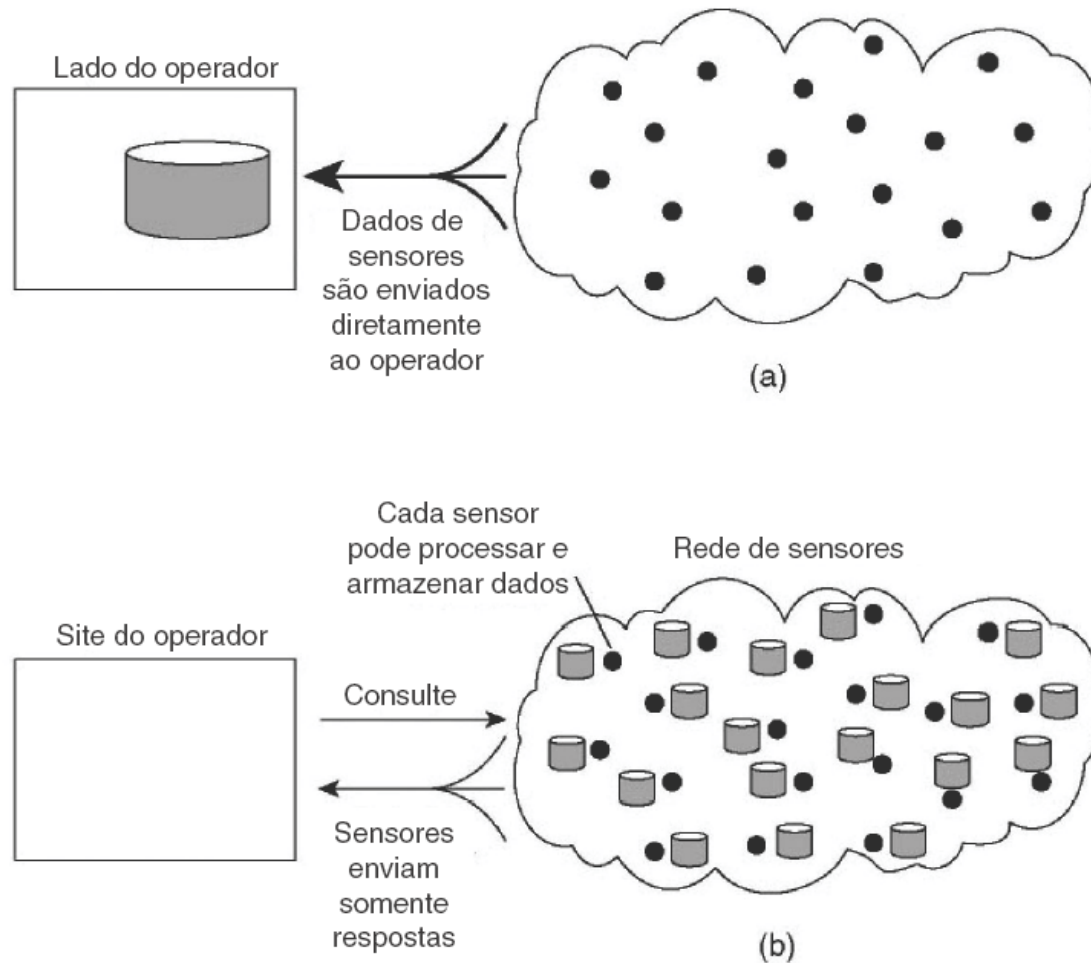
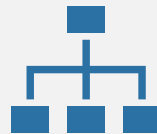


Figura 1.10 Organizando um banco de dados de rede de sensores e, ao mesmo tempo, armazenando e processando dados (a) somente no site do operador ou (b) somente nos sensores.

Redes de Sensores



Como montar dinamicamente uma árvore eficiente em uma rede de sensores?

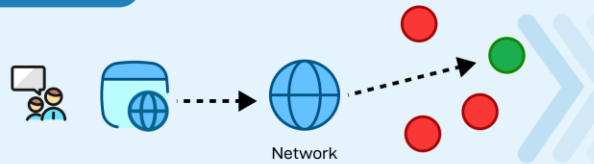


Como ocorre a agregação de resultados? Elas pode ser controlada?



O que acontece quando enlaces de redes falham?

Unicast



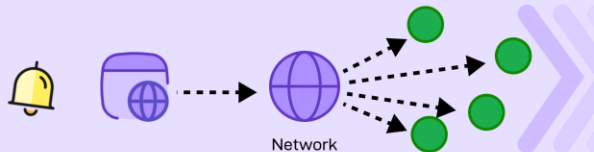
Unique Sender and Single Receiver

Uses a unique destination IP address
Routers look up the routing table to forward



Unicast is used in HTTP, FTP, SMTP

Broadcast



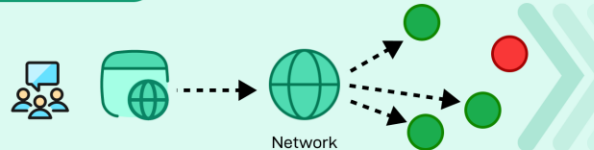
Single Sender and Multiple Receivers

Uses a reserved destination address
Usually restricted to a single logical network



Broadcast is used in ARP, DHCP, NTP

Multicast



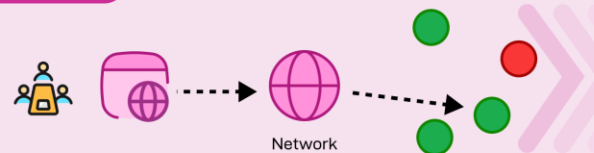
Sender to Specific Receivers

Uses a unique destination IP address
Routers look up the routing table to forward



Multicast is used Video conference apps

Anycast



Sender to Single Representative of a Group

Anycast is an extension of unicast
Same unicast address is assigned to multiple devices



Anycast is used in DNS, CDNs

Conclusão



1. **Sistemas distribuídos são a norma** — a maioria dos sistemas modernos (nuvem, mobile, web) é distribuída, mesmo quando essa distribuição fica oculta do usuário via transparência.
2. **Escalabilidade se constrói com um conjunto de técnicas** — particionamento, replicação, cache e balanceamento de carga trabalham juntos; não existe uma solução única.
3. **Descentralização é o caminho natural em larga escala** — coordenadores centrais e estado global não escalam; por isso DHTs, consenso distribuído e P2P são fundamentais.
4. **As falácias de Peter Deutsch continuam atuais** — assumir que a rede é confiável, rápida e segura é a raiz de boa parte dos problemas de projeto em sistemas reais.

Obrigado pela atenção!

Prof. Paulo Bressan